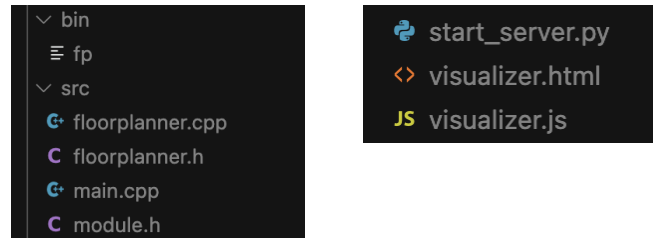


Physical Design for Nanometer ICs, Spring 2025

Programming Assignment #2 R13943074 楊珩

1. File and data structure description



I used the data structure from the sample code with some slight modifications (which will be detailed below), and I also added some web-based GUI files, allowing users to observe the floorplan results directly on the webpage.

(a) module.h

```
> class Contour ...  
> class Terminal ...  
> class Block : public Terminal ...  
> class Net ...  
> struct BStarTreeNode { ...  
> class BStarTree { ...
```

Below is a brief explanation of five classes and structs:

- **Contour**: Stores the nodes of the Y-contour, including `prev`, `next`, and the (x, y) coordinates. Implementation details will be introduced later.
- **Terminal**: Records the coordinates of a terminal, as well as its position after being moved to the fixed outline.
- **Block**: Similar to **Terminal**, but each **Block** is also associated with a **Contour** member. It is used to update the Y-contour during packing and also keeps track of rotation information.
- **Net**: Records all interconnected terminals and blocks, and is used for calculating HPWL (Half-Perimeter Wire Length).
- **BStarTreeNode**: Each node contains a **Block** member, as well as a `parent`, `left child`, and `right child`.
- **BStarTree**: Holds the root node (`BStarTreeNode* _root`) and includes several helper functions to retrieve information about the tree.

(b) floorplan.h

method:

```
void parseInput(fstream& input_blk, fstream& input_net);
void determineSide();
void adjustTerminals();
void generateBalancedBStarTree();
void floorplan();
void writeResult(fstream& output);
void calculateTotalBoundingBox();
void calculateTotalHPWL();
void calculatePenalty();
void calculateCost();
void exportJSON(const string& filename);
void updatePosition(BStarTreeNode* node);
void updateTreePositions(BStarTreeNode* node);
void simulatedAnnealing();
void rotateBlock(Block* block);
void deleteAndInsertNode(BStarTreeNode* node, BStarTreeNode* to_node);
void swapNodes(BStarTreeNode* node1, BStarTreeNode* node2);
double getRandomDouble();
bool acceptNewSolution(double oldCost, double newCost, double temperature);
void keepRotate();
void shuffleBlocks();
void makeSymmetricTerminals();
void makeSymmetricBlocks();
void returnOriginalTerminals();
```

data member:

```
// instance
vector<Block*> _blocks;
vector<Terminal*> _terminals;
vector<Net*> _nets;

// basic information
size_t _max_x;           // maximum x coordinate for all blocks
size_t _max_y;           // maximum y coordinate for all blocks
bool _xSymmetric;        // whether the floorplan is x-symmetric
bool _ySymmetric;        // whether the floorplan is y-symmetric
double _runtime;         // runtime of the floorplan

// about tree
BStarTree _bstar_tree;   // current tree
BStarTree _best_tree;    // best tree found so far
BStarTree _last_tree;    // record the last tree

// about cost
double _cost;            // cost of the floorplan
double _init_cost;       // initial cost of the floorplan
double _best_cost;       // best cost found so far
double _alpha;           // scaling factor
double _total_area;      // total area of the floorplan
double _total_hpwl;      // total hpwl of the floorplan
double _avg_area;        // average area of the floorplan
double _avg_hpwl;        // average hpwl of the floorplan
double _best_area;       // best area found so far
double _best_hpwl;       // best hpwl found so far
double _penalty;         // penalty for the floorplan
double _best_penalty;    // best penalty found so far

// about SA
int _accept_num;         // number of accepted moves when new_cost > old_cost
double _current_temp;    // current temperature
```

2. Function description

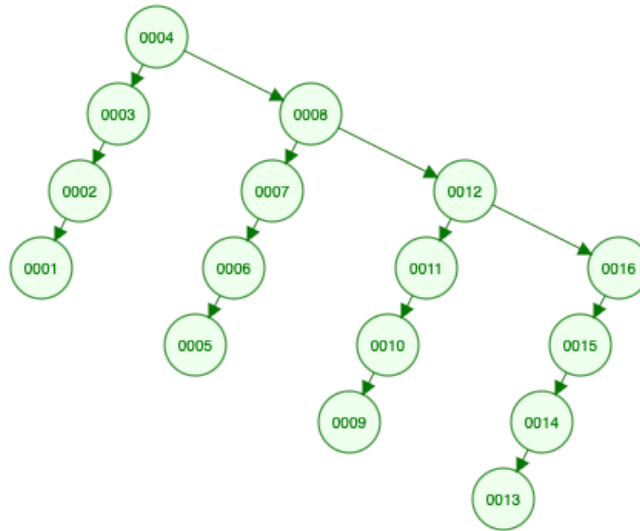
determineSide, adjustTerminals

Based on the distribution of the terminals in the case, I determine which region their centroid is closest to — top-left, bottom-left, top-right, or bottom-right — and use the corresponding corner of the bounding box as the starting point for the B*-tree.

Furthermore, if a terminal is located outside the outline, its position will be moved to the edge of the outline. This ensures that when placing blocks, there is no need to deal with the trade-off between penalty and net cost.

generateBalancedBStarTree

Initially, all the blocks are arranged into a B*-tree in the form shown in the figure below, where the overall “width” and “height” of the tree structure should be approximately equal, which helps achieve a relatively good initial placement.



calculateTotalBoundingBox, calculateTotalHPWL, calculatePenalty

`CalculateCost()` refers to the cost used by the Simulated Annealing function. It includes the bounding box area, HPWL, and a penalty for exceeding the outline. The penalty is calculated based on the area that exceeds the boundary, rather than using the squared difference of the aspect ratio. The formula I used is as follows, where `_avg_hpwl` is the HPWL obtained from the initial placement, and `_avg_area` is the area of the outline.

```
_cost = (_alpha) * (_total_area / _avg_area) + (1 - _alpha) * (_total_hpwl / _avg_hpwl) + (_penalty * 100 / _avg_area);
```

updateTreePositions

After each update to the tree structure, the new tree needs to be packed, and this function is used to update the position of each block accordingly.

keepRotate

During the SA process, if the best tree result is updated, the rotation state of each block at that time should also be recorded; otherwise, it will be overwritten later.

makeSymmetricTerminals, makeSymmetricBlocks, returnOriginalTerminals

If the coordinates were flipped during the initial `adjustTerminals` step, remember to flip the coordinates of both the blocks and terminals back at the end. Additionally, terminals that were snapped to the outline boundary should be restored to their original positions.

3. Algorithm (pseudo code)

Contour (double linked list)

During the packing implementation, it is necessary to maintain the contour. To achieve this, each block has a `contour` member.

```
class Contour
{
public:
    // Constructor and destructor
    Contour(size_t x, size_t y) :
        _x(x), _y(y), _prev(NULL), _next(NULL) { }
    ~Contour() { }

    // Basic access methods
    size_t getX() const { return _x; }
    size_t getY() const { return _y; }
    Contour* getPrev() const { return _prev; }
    Contour* getNext() const { return _next; }
    string getName() const { return _name; }

    // Set functions
    void setX(size_t x) { _x = x; }
    void setY(size_t y) { _y = y; }
    void setPrev(Contour* prev) { _prev = prev; }
    void setNext(Contour* next) { _next = next; }
    void setName(string& name) { _name = name; }

private:
    size_t _x; // x coordinate of the contour
    size_t _y; // y coordinate of the contour
    Contour* _prev; // pointer to the previous node
    Contour* _next; // pointer to the next node
    string _name; // name of the contour
};
```

The contour stores the x and y positions, which correspond to the top-right corner coordinates of each block.

When placing a block, if it is the right child of its parent, the x-coordinate should be placed immediately to the right of the parent. If it is the left child, the x-coordinate should be the same as the parent's.

As for the y-coordinate, it is recorded using the contour. The largest y-contour within the x-range is found and used for placement. After placing the block, the current block's contour node is connected to the existing doubly linked list.

Case1: if the block is the left child of its parent

```
// 檢查水平輪廓線
Contour* current = node->parent->block->getContour();
int count = 0;
while (current->getNext() != nullptr) {
    current = current->getNext();
    if (count == 0) {
        y = current->getY();
    }
    else {
        y = max(y, current->getY());
    }
    count++;
    if (current->getX() >= x + node->block->getWidth()) {
        node->block->getContour()->setNext(current);
        current->setPrev(node->block->getContour());
        break;
    }
}
node->block->getContour()->setX(x + node->block->getWidth());
node->block->getContour()->setY(y + node->block->getHeight());
node->block->getContour()->setPrev(node->parent->block->getContour());
node->parent->block->getContour()->setNext(node->block->getContour());
node->block->setPos(x, y, x+node->block->getWidth(), y+node->block->getHeight());
```

Case2: if the block is the right child of its parent

```
// 檢查水平輪廓線
Contour* current = node->parent->block->getContour();
if (current->getX() >= x + node->block->getWidth()) {
    node->block->getContour()->setNext(current);
    current->setPrev(node->block->getContour());
}
else {
    while (current->getNext() != nullptr) {
        current = current->getNext();
        y = max(y, current->getY());
        if (current->getX() >= x + node->block->getWidth()) {
            node->block->getContour()->setNext(current);
            current->setPrev(node->block->getContour());
            break;
        }
    }
}
node->block->getContour()->setX(x + node->block->getWidth());
node->block->getContour()->setY(y + node->block->getHeight());
node->block->setPos(x, y, x+node->block->getWidth(), y+node->block->getHeight());
```

Simulated Annealing

Following the standard approach in the lecture notes, downhill moves are always accepted, and uphill moves are accepted with a certain probability.

```
// about SA
double temperature;
double _inital_temperature = 100;
double _final_temperature = 0.1;
double _cooling_rate = 0.98;
int iteration_per_temperature = 5000;

double probability = exp((oldCost - newCost) * 10000 / temperature);
```

Pseudo code of floorplan

```
void FloorPlanner::floorplan() {
    // set terminal placement
    adjustTerminals();
    // initial placement
    generateBalancedBStarTree();
    // SA
    while (current_temp > final_temp) {
        for (int i = 0; i < iterations_per_temp; i++) {
            // keep current placement
            keep_current_placement();

            // select random operation
            switch (operation) {
                case 0:
                    rotateBlock(block);
                    break;
                case 1:
                    deleteAndInsertNode(block1, block2);
                    break;
                case 2:
                    swapBlock(block1, block2);
                    break;
            }

            // update tree positions
            updateTreePositions();
            // calculate cost
            calculateCost();
            if (newCost < oldCost) {
                oldCost = newCost;
            } else {
                if (randNum < probability) uphill();
                else reject();
            }
            if (newCost < bestCost) {
                // keep best placement
                updateBestPlacement();
                updateBestRotation();
            }
        }

        // update temperature
        temperature *= _cooling_rate;
    }

    // restore best placement
    restoreBestPlacement();
    restoreBestRotation();
    // make symmetry
    makeSymmetryTerminals();
    makeSymmetryBlocks();
    // return original terminal placement
    return_original_terminal_placement();
}
```

4. Findings

Starting Point of the B-Tree

In the lecture's algorithm, the B*-tree always starts at (0, 0). However, I noticed that in these cases, when displaying the terminals using the GUI, they are located far from the outline and mostly clustered toward the top-right corner. Therefore, I considered building the B*-tree starting from the top-right corner.

But since this would affect the contour algorithm and the x, y coordinates, I instead chose to mirror the terminals at the beginning, so that the B*-tree could still start at (0, 0). After all the steps are completed, I mirror both the blocks and terminals back to their original orientation.

This approach significantly improved the net's HPWL cost.

GUI Interface

Before writing the code, I felt it was important to first develop a GUI that would allow me to intuitively observe the current block placements and terminal positions. This would be very helpful for debugging and improving the algorithm, as it allows me to clearly identify where the problems are.

So I created a web interface that, after the floorplanning is completed, outputs a JSON file. The webpage can read this JSON file and visualize the actual floorplan layout. This saved me a lot of time during the initial debugging phase.

5. Results (for case alpha = 0.5)

Case	Area	Wirelength	Time (s)	Cost	Quality Score	Time Score	Total Score
ami33	1206576	75745	3.45	0.0430646	8.128918	5.588819	7.62089
ami49	37879744	762299	6.787	-0.0119837	9.5934214	6.3772679	8.95019
apte	47313280	708200	0.088	0.0425583	10.55986	7.636442	9.97518
hp	9707880	211837	0.086	-0.114873	32.7295	7.466790	27.67696
xerox	20424474	498367.5	0.094	0.0417769	8.9546628	7.881427	8.74002

Case ami33:

```
Congradulations, your results are correct

[Eval] Your area & wirelength: 1206576.000000 & 75745.000000
[Eval] Your runtime: 3.395000
[Eval] Your normalized cost = 0.5*(A-minA)/(maxA-minA) + 0.5*(WL-minWL)/(maxWL-minWL) = 0.0430646
[Eval] SCORE = QUALITY_SCORE*0.8 + RUNTIME_SCORE*0.2
           = 8.128918*0.8 + 5.591194*0.2
           = 7.62137
```

Case ami49:

```
Congradulations, your results are correct

[Eval] Your area & wirelength: 37879744.000000 & 762299.000000
[Eval] Your runtime: 6.651000
[Eval] Your normalized cost = 0.5*(A-minA)/(maxA-minA) + 0.5*(WL-minWL)/(maxWL-minWL) = -0.0119837
[Eval] SCORE = QUALITY_SCORE*0.8 + RUNTIME_SCORE*0.2
      = 9.5934214*0.8 + 6.3877519*0.2
      = 8.95229
```

Case apte:

```
Congradulations, your results are correct

[Eval] Your area & wirelength: 47313280.000000 & 708200.000000
[Eval] Your runtime: 0.089000
[Eval] Your normalized cost = 0.5*(A-minA)/(maxA-minA) + 0.5*(WL-minWL)/(maxWL-minWL) = 0.0425583
[Eval] SCORE = QUALITY_SCORE*0.8 + RUNTIME_SCORE*0.2
      = 10.55986*0.8 + 7.626369*0.2
      = 9.97316
```

Case hp:

```
Congradulations, your results are correct

[Eval] Your area & wirelength: 9707880.000000 & 211837.000000
[Eval] Your runtime: 0.086000
[Eval] Your normalized cost = 0.5*(A-minA)/(maxA-minA) + 0.5*(WL-minWL)/(maxWL-minWL) = -0.114873
[Eval] SCORE = QUALITY_SCORE*0.8 + RUNTIME_SCORE*0.2
      = 32.7295*0.8 + 7.466790*0.2
      = 27.67696
```

Case xerox:

```
Congradulations, your results are correct

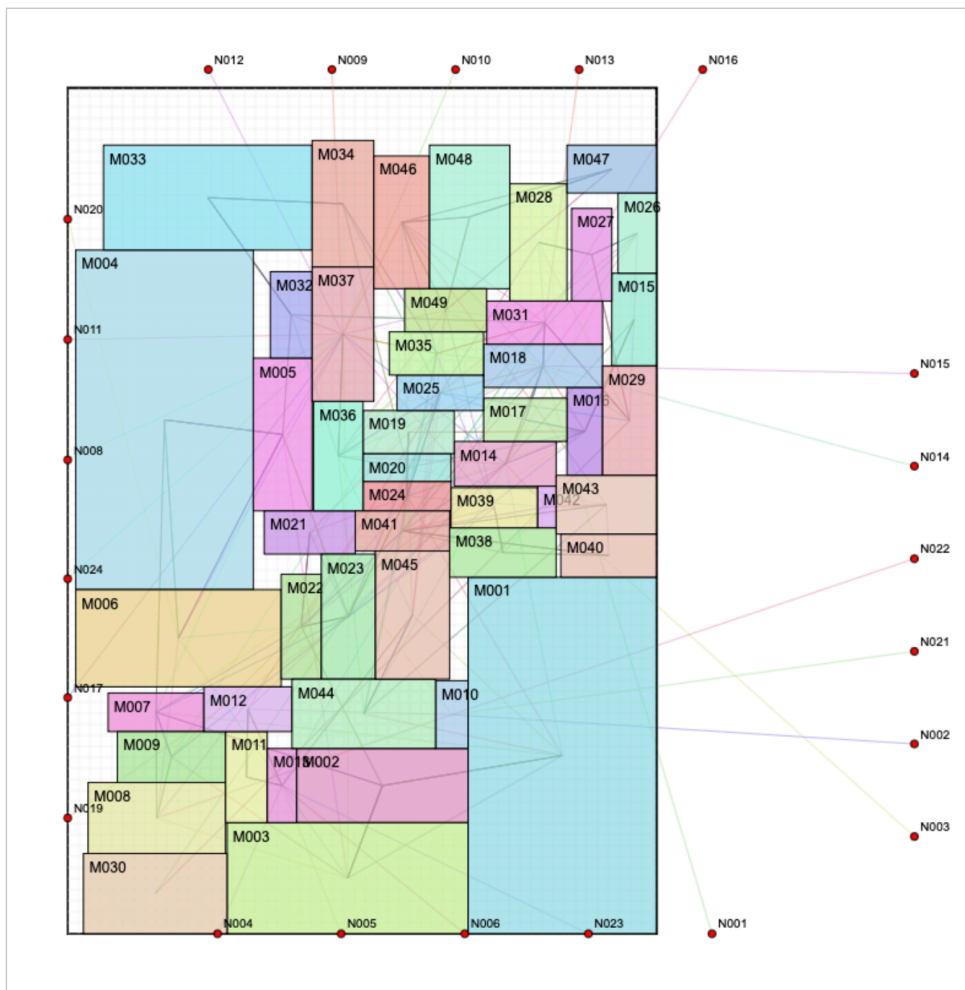
[Eval] Your area & wirelength: 20424474.000000 & 498367.500000
[Eval] Your runtime: 0.124000
[Eval] Your normalized cost = 0.5*(A-minA)/(maxA-minA) + 0.5*(WL-minWL)/(maxWL-minWL) = 0.0417769
[Eval] SCORE = QUALITY_SCORE*0.8 + RUNTIME_SCORE*0.2
      = 8.9546628*0.8 + 7.578444*0.2
      = 8.67942
```


GUI Interface

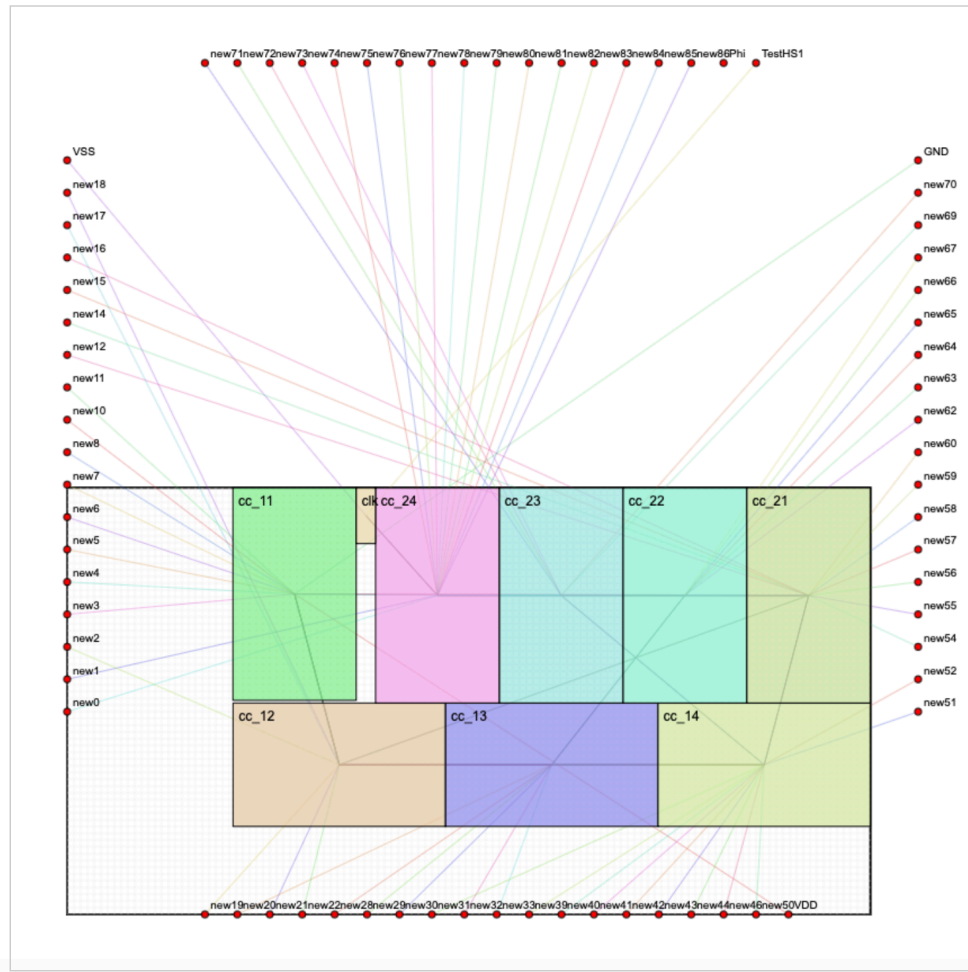
Current floorplan: ami33



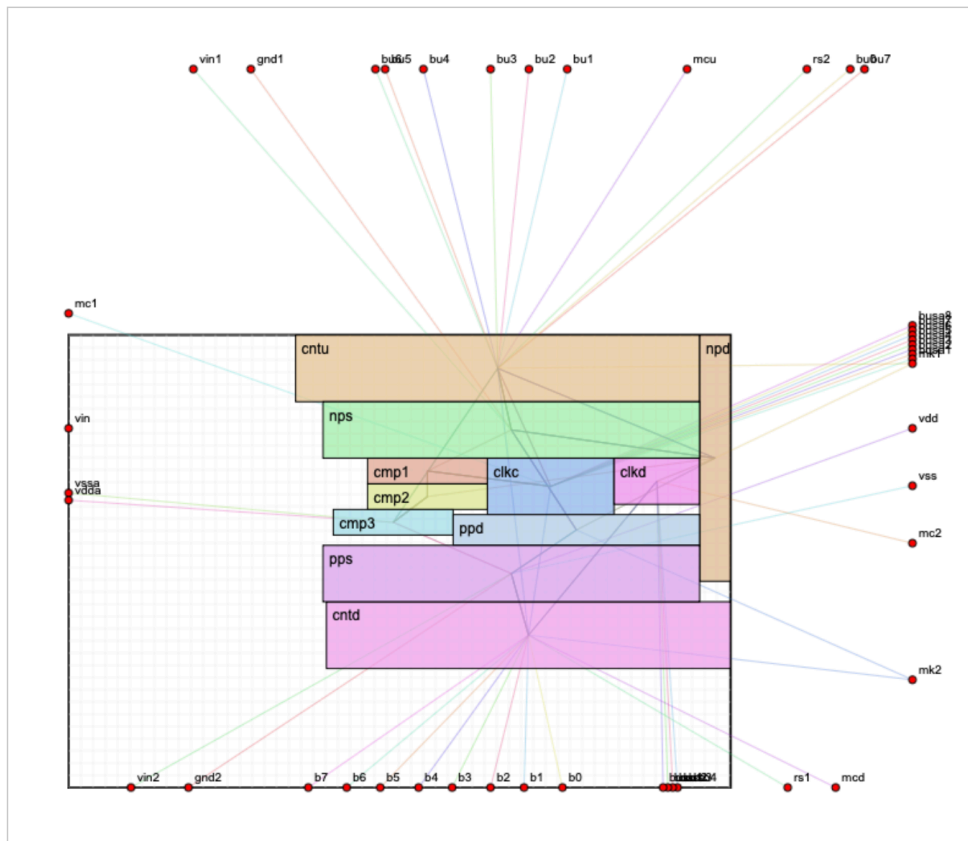
Current floorplan: ami49



Current floorplan: apte



Current floorplan: hp



Current floorplan: xerox

